

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	K Madan Mohan Reddy	Reg. No.:	192411206
Date:	20-8-2026	Duration:	60 minutes

Code of Conduct

I K Madan Mohan Reddy (192411206) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Byreddy_____

Annexure A – Git & Docker Technical Assignment

"Online Loan Eligibility Prediction and Approval Management Platform"

1. PROBLEM ANALYSIS

1.1 Problem Statement

Traditional loan approval processes often require applicants to visit banks or financial institutions, submit multiple documents, and wait several days for eligibility verification and approval. Manual evaluation of factors such as income, employment status, credit history, existing loans, and repayment capacity can be time-consuming and may lead to inconsistent decisions.

The Online Loan Eligibility Prediction and Approval Management Platform is proposed to automate and simplify the loan application and approval process. The system allows users to submit their personal, financial, employment, and loan-related information online.

- A machine learning model analyzes the applicant's information and predicts whether the applicant is eligible or not eligible for the requested loan.
- The platform can also provide loan officers or administrators with a centralized dashboard to review applications, verify submitted information, approve or reject applications, and track application status. The system maintains application records and generates reports to support efficient loan management.

1.2 Objectives

The main objectives of the Campus Placement and Recruitment Portal are:

- To develop a centralized platform for campus recruitment activities.
- To allow students to create and maintain placement profiles.
- To provide students with information about available job opportunities.
- To allow eligible students to apply for suitable job positions.
- To help placement officers manage students, companies, and recruitment activities.
- To allow recruiters to post job vacancies and manage applications.
- To maintain recruitment and placement records systematically.
- To reduce manual paperwork and improve communication.
- To provide a reliable and accessible recruitment management system.
- To improve the overall efficiency of the campus placement process.

1.3 Functional Requirements

Module	Functional Requirements
Student Module	Registration, login, profile management, academic details, job search, eligibility checking, job application, application tracking
Recruiter Module	Registration, login, company profile management, job posting, eligibility criteria, application viewing, candidate shortlisting
Placement Officer Module	Student management, company management, job-post management, application monitoring, placement-drive management, reports

Student Module

- Student registration and login.
- Student profile creation and updating.
- Academic details management.
- Viewing available job opportunities.
- Checking job eligibility.
- Applying for jobs.

- Viewing application status.
- Viewing placement results.

Recruiter Module

- Recruiter and company registration.
- Company profile management.
- Posting job vacancies.
- Defining eligibility criteria.
- Viewing student applications.
- Shortlisting candidates.
- Updating recruitment status.

Placement Officer Module

- Placement officer login.
- Managing student records.
- Managing recruiter and company information.
- Managing job postings.
- Monitoring applications.
- Managing placement drives.
- Maintaining placement records.
- Generating recruitment reports.

1.4 Non-Functional Requirements

Requirement	Description
Security	Protect user information through authentication and authorization.
Performance	Provide efficient responses to user requests.
Scalability	Support an increasing number of students and recruiters.
Reliability	Provide consistent and dependable access.
Usability	Provide a simple and user-friendly interface.
Maintainability	Keep the application organized and easy to modify.
Portability	Allow the application to run consistently using Docker.

1.5 Expected Outcomes

The expected outcome is a functional web-based Campus Placement and Recruitment Portal that provides a centralized system for managing campus recruitment activities.

The system should allow:

- Students to search and apply for job opportunities.
- Recruiters to manage vacancies and applications.
- Placement officers to monitor recruitment activities.
- The system to maintain student, company, job, and application records.
- The application to be deployed consistently using Docker.

2. PROJECT STRUCTURE DESIGN

2.1 Repository Structure

campus-placement-portal/

```
|
|— frontend/
|   |— index.html
|   |— login.html
|   |— register.html
|   |— student-dashboard.html
|   |— recruiter-dashboard.html
|   |— admin-dashboard.html
|   |— jobs.html
|   |— profile.html
|   |— css/
|       |— style.css
|
|— backend/
|   |— server.js
|   |— routes/
|       |— auth.js
|       |— students.js
|       |— recruiters.js
|       |— jobs.js
|       |
|   |— controllers/
```

```

| | | └── authController.js
| | | └── studentController.js
| | | └── recruiterController.js
| | | └── jobController.js
| |
| | └── config/
| |     └── database.js
|
| └── database/
|     └── schema.sql
|
| └── Dockerfile
| └── docker-compose.yml
| └── .gitignore
| └── package.json
└── README.md

```

2.2 Folder and File Description

Folder/File	Purpose
frontend/	Contains the user interface of the application.
backend/	Contains server-side application logic.
routes/	Contains API routes and endpoints.
controllers/	Contains application processing logic.
config/	Contains configuration files.
database/	Contains database schema and SQL files.
Dockerfile	Contains instructions for creating the Docker image.
docker-compose.yml	Defines and manages application services.
.gitignore	Specifies files that should not be tracked by Git.
package.json	Contains project dependencies and scripts.
README.md	Contains project information and setup instructions.

3. GIT REPOSITORY INITIALIZATION

Git is a distributed version control system used to track changes made to the Campus Placement and Recruitment Portal.

3.1 Repository Initialization

The project directory is first created using:

```
mkdir campus-placement-portal  
cd campus-placement-portal
```

Git is initialized using:

```
git init
```

The repository status is checked using:

```
git status
```

The project files are added using:

```
git add .
```

The initial commit is created using:

```
git commit -m "Initial project setup"
```

3.2 Essential Git Files

`.git`

- The `.git` directory is created automatically when `git init` is executed.
- It contains Git metadata.
- It stores commit history, branches, and repository configuration.

`.gitignore`

- Prevents unnecessary or sensitive files from being tracked.
- Helps keep the repository clean.

Example:

```
node_modules/  
.env  
*.log  
.DS_Store
```

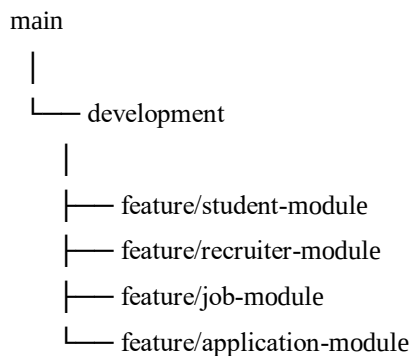
README.md

- Provides project information.
- Contains installation instructions.
- Describes technologies used.
- Provides Git and Docker commands.

4. GIT BRANCHING STRATEGY

A structured branching strategy is used to improve collaboration and maintain stable project versions.

4.1 Branching Structure



4.2 Main Branch

- Contains the stable version of the application.
- Only tested and approved code should be merged into this branch.

4.3 Development Branch

- Used to combine different features.
- Allows testing before code is moved to the main branch.

4.4 Feature Branches

Feature branches are created for individual modules.

Examples:

```
git checkout -b feature/student-module
git checkout -b feature/recruiter-module
git checkout -b feature/job-module
```

4.5 Release Branch

A release branch can be created when the application is ready for final testing.

```
git checkout -b release/v1.0
```

4.6 Hotfix Branch

A hotfix branch can be created to fix critical problems in the released version.

```
git checkout -b hotfix/login-fix
```

4.7 Branching Strategy Table

Branch	Purpose
main	Stable production version
development	Integration and testing
feature/student-module	Student functionality
feature/recruiter-module	Recruiter functionality
feature/job-module	Job management
feature/application-module	Job application functionality
release/v1.0	Final testing before release
hotfix/login-fix	Critical issue correction

This branching strategy is appropriate because developers can work independently on different modules without directly affecting the stable version.

5. VERSION CONTROL WORKFLOW

The Git workflow consists of creating branches, developing features, committing changes, merging branches, and synchronizing the repository.

5.1 Git Operations

Operation	Command	Purpose
-----------	---------	---------

Initialize	git init	Creates a Git repository.
Check status	git status	Displays repository status.
Add files	git add .	Stages files.
Commit	git commit -m "message"	Records changes.
Create branch	git checkout -b branch-name	Creates and switches to a branch.
Switch branch	git checkout branch-name	Switches to an existing branch.
Merge	git merge branch-name	Combines branch changes.
Push	git push	Uploads changes to remote repository.
Pull	git pull	Retrieves remote changes.
View history	git log --oneline	Displays commit history.

5.2 Create Development Branch

git checkout -b development

5.3 Create Feature Branch

git checkout -b feature/student-module

5.4 Add Files

git add .

5.5 Commit Changes

git commit -m "Add student registration module"

5.6 Merge Feature

git checkout development

git merge feature/student-module

5.7 Push Changes

git push origin development

5.8 Pull Changes

git pull origin development

5.9 Conflict Resolution

If two developers modify the same section of a file, Git may produce a merge conflict.

The conflict should be resolved manually by:

- Opening the conflicting file.
- Identifying the conflicting sections.
- Selecting the correct code.

- Removing conflict markers.
- Saving the file.
- Adding the corrected file.
- Creating a new commit.

```
git add .
```

```
git commit -m "Resolve merge conflict"
```

6. COMMIT HISTORY ANALYSIS

Meaningful commit messages help developers understand the changes made to the project.

6.1 Example Commit History

Initial project setup

Add database schema

Add student registration module

Add student login module

Add recruiter module

Add job posting functionality

Add job application functionality

Add admin dashboard

Add Docker configuration

Fix database connection issue

Resolve merge conflict

Prepare version 1.0 release

The commit history can be displayed using:

```
git log --oneline
```

6.2 Good Commit Message Practices

- Use short and meaningful messages.
- Clearly describe the change.
- Focus on one logical change per commit.
- Avoid vague messages.
- Maintain consistent naming.
- Use commits to create a clear development history.

Example:

```
git commit -m "Add student registration module"
```

is better than:

```
git commit -m "changes"
```

A well-maintained commit history improves:

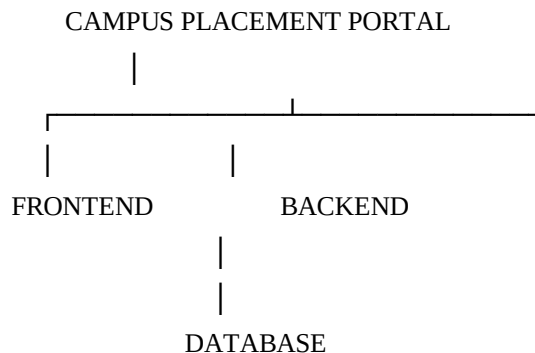
- Project maintenance.
- Debugging.
- Collaboration.
- Version tracking.
- Error recovery.

7. DOCKER ENVIRONMENT DESIGN

Docker is a containerization platform that packages an application and its dependencies into isolated containers.

Docker is suitable for the Campus Placement and Recruitment Portal because the application contains multiple components such as the frontend, backend, and database.

7.1 Application Components



The major services are:

- Frontend.
- Backend.
- Database.

7.2 Docker Benefits for the Project

- Provides environment consistency.
- Simplifies deployment.
- Improves portability.
- Provides application isolation.
- Simplifies dependency management.
- Supports scalability.
- Makes development setup easier.

8. DOCKERFILE DEVELOPMENT

A Dockerfile contains instructions required to build the application container.

8.1 Dockerfile

```
FROM node:20
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

8.2 Dockerfile Instruction Table

Instruction	Purpose
FROM	Specifies the base image.
WORKDIR	Sets the working directory.
COPY	Copies files into the container.
RUN	Executes commands during image creation.
EXPOSE	Specifies the application port.
CMD	Starts the application when the container runs.

9. DOCKER COMPOSE DESIGN

Docker Compose is used to define and manage multiple containers as a single application.

9.1 Docker Compose Configuration

services:

backend:

build: .

container_name: placement_backend

ports:

- "3000:3000"

environment:

DB_HOST: database

DB_USER: placement_user

DB_PASSWORD: placement_password

DB_NAME: placement_db

depends_on:

- database

database:

image: mysql:8.0

container_name: placement_database

environment:

- "3306:3306"

volumes:

- placement_data:/var/lib/mysql

volumes:

placement_data:

9.2 Service Description

Service	Technology	Purpose	Port
Backend	Node.js	Runs application server and APIs.	3000
Database	MySQL	Stores portal information.	3306

9.3 Environment Variables

Environment variables are used to provide database configuration information.

They include:

- Database host.
- Database username.
- Database password.
- Database name.

Using environment variables helps prevent configuration information from being directly written into the application source code.

9.4 Volumes

The database uses:

volumes:

- placement_data:/var/lib/mysql

The volume provides persistent storage for database information.

9.5 Dependencies

The backend depends on the database service:

depends_on:

- database

This defines the relationship between the services.

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Build Docker Image

`docker build -t campus-placement-portal .`

10.2 Start Containers

`docker compose up -d`

10.3 Check Running Containers

`docker ps`

10.4 View Logs

`docker compose logs`

10.5 Stop Containers

`docker compose down`

10.6 Testing Procedure

Test No.	Test Case	Expected Result
1	Start Docker Desktop	Docker starts successfully.
2	Build Docker image	Image is created successfully.
3	Start Docker Compose	Required containers start.
4	Execute <code>docker ps</code>	Containers are displayed as running.
5	Open application	Portal loads successfully.
6	Student registration	Student account is created.
7	Student login	Student dashboard opens.
8	View jobs	Available jobs are displayed.
9	Apply for job	Application is recorded.
10	Recruiter login	Recruiter dashboard opens.
11	Post job	Job vacancy is created.
12	Database verification	Data is stored successfully.

10.7 Expected Result

The Campus Placement and Recruitment Portal should successfully run inside Docker containers.

The expected results are:

- Backend container runs successfully.
- Database container runs successfully.
- Backend communicates with the database.
- Students can access the portal.
- Recruiters can manage job postings.
- Applications can be stored.
- Recruitment information can be retrieved successfully.

11. BENEFITS OF GIT AND DOCKER

11.1 Benefits of Git

Collaboration

- Allows multiple developers to work on different modules.
- Supports feature branches.
- Allows changes to be merged systematically.

Version Management

- Maintains complete project history.
- Tracks modifications.
- Allows previous versions to be restored.

Branching

- Allows independent feature development.
- Prevents unstable code from affecting the main branch.

Code Review

- Changes can be reviewed before being merged.
- Helps maintain code quality.

11.2 Benefits of Docker

Area	Benefit
Portability	Application can run consistently across different systems.
Deployment	Simplifies application deployment.
Dependency Management	Packages application dependencies.
Scalability	Additional containers can be deployed when required.
Maintainability	Services can be managed independently.
Environment Consistency	Reduces differences between development and deployment environments.

11.3 Overall Benefits

The combination of Git and Docker provides:

- Better collaboration.
- Effective version management.
- Portable deployment.
- Consistent environments.
- Easier maintenance.
- Improved scalability.
- Faster deployment.
- Better project organization.

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges

Git Merge Conflicts

- Conflicts may occur when multiple developers modify the same file.
- Proper conflict-resolution procedures are required.

Branch Management

- Managing multiple branches can become difficult as the project grows.
- A clear branching strategy is required.

Docker Configuration

- Incorrect ports or environment variables can cause container failures.
- Docker configuration must be tested carefully.

Database Connectivity

- The backend must correctly connect to the MySQL container.
- Correct service names and environment variables must be used.

Dependency Management

- Incorrect package versions can cause application failures.
- Dependencies should be maintained properly.

Resource Usage

- Running multiple containers requires system resources.
- Containers should be optimized when necessary.

12.2 Solutions

- Use a proper Git branching strategy.
- Commit changes regularly.
- Use meaningful commit messages.
- Pull the latest changes before development.
- Resolve conflicts carefully.
- Maintain a proper .gitignore file.
- Use environment variables for configuration.
- Use Docker Compose for service management.
- Use persistent database volumes.
- Test containers before deployment.

12.3 Future Enhancements

S.No.	Future Enhancement
1	AI-based candidate-job matching

2	Automated resume screening
3	Email and notification services
4	Online aptitude and coding tests
5	Interview scheduling
6	Video interview integration
7	Placement analytics dashboard
8	Automated eligibility checking
9	Cloud deployment
10	Role-based access control
11	CI/CD pipeline
12	Kubernetes-based deployment

CONCLUSION

The Campus Placement and Recruitment Portal provides a centralized solution for managing campus recruitment activities involving students, recruiters, and placement officers. The system reduces manual work and provides an organized platform for managing student profiles, job opportunities, applications, recruitment activities, and placement records.

Git provides effective version control, collaboration, branching, and change management throughout the development process. The use of separate branches allows developers to work independently on different modules while maintaining a stable main version of the project.

Docker provides a consistent and portable environment for deploying the application and its dependencies. Docker Compose allows multiple services, such as the backend and database, to be managed together.

The combination of Git and Docker improves the development, testing, deployment, scalability, and maintainability of the Campus Placement and Recruitment Portal. The system can be further enhanced through AI-based recruitment, automated resume screening, analytics, CI/CD, cloud deployment, and container orchestration technologies.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

